

第 12 章：强化学习面试速查与项目准备

本章符号说明

本章是速查表，会集中复现前面章节的核心符号：

符号/缩写	English	中文	含义
MDP	Markov Decision Process	马尔可夫决策过程	RL 标准建模。
$S / A / R$	State / Action / Reward	状态 / 动作 / 奖励	MDP 三个基础元素。
G_t	Return	回报 / 累计折扣奖励	从时刻 t 开始的累计收益。
$V_{\pi}(s)$	State-value Function	状态价值函数	策略 π 下状态 s 的长期价值。
$Q_{\pi}(s,a)$	Action-value Function	动作价值函数	策略 π 下状态-动作对 (s,a) 的长期价值。
$A_{\pi}(s,a)$	Advantage Function	优势函数	动作相对状态平均价值的优势；这里的 A 不是 action space。
$J(\theta)$	Objective Function	目标函数	policy optimization 中要最大化的目标。
$L(\theta)$	Loss Function	损失函数	神经网络训练中要最小化的目标。
KL	Kullback-Leibler Divergence	KL 散度	衡量两个策略分布的差异。
DQN / PPO / SAC	Deep Q-Network / Proximal Policy Optimization / Soft Actor-Critic	深度 Q 网络 / 近端策略优化 / 软 Actor-Critic	面试中最高频的三类 deep RL 算法。
RL / RLHF	Reinforcement Learning / Reinforcement Learning from Human Feedback	强化学习 / 基于人类反馈的强化学习	本套课程和对齐流程中的核心概念。
DP / MC / TD	Dynamic Programming / Monte Carlo / Temporal Difference	动态规划 / 蒙特卡洛 / 时序差分	价值估计和控制的基础方法。
SARSA	State-Action-Reward-State-Action	状态-动作-奖励-状态-动作	on-policy TD control 算法。

符号/缩写	English	中文	含义
REINFORCE	REINFORCE algorithm	REINFORCE 算法	Monte Carlo policy gradient 基线算法。
A2C / A3C	Advantage Actor-Critic / Asynchronous Advantage Actor-Critic	优势 Actor-Critic / 异步优势 Actor-Critic	actor-critic 的同步和异步版本。
GAE	Generalized Advantage Estimation	广义优势估计	PPO 常用的 advantage 估计方法。
DDPG / TD3	Deep Deterministic Policy Gradient / Twin Delayed DDPG	深度确定性策略梯度 / 双延迟 DDPG	连续动作控制算法。
UCB	Upper Confidence Bound	置信上界	bandit 中基于不确定性 bonus 的探索方法。
MLP	Multi-Layer Perceptron	多层感知机	CartPole DQN 项目中常用的小型神经网络。
PG	Policy Gradient	策略梯度	直接优化策略参数的一类方法。

本章目标

本章用于面试前快速复习。你需要整理：

- 核心概念速查。
- 常见算法横向对比。
- 高频公式。
- 高频问答。
- 项目表达模板。

本章主线

本章不是引入新算法，而是把前面所有内容压缩成面试表达结构。复习顺序是：先用一张图串起 MDP、Bellman、DP、MC/TD、DQN、Policy Gradient、Actor-Critic、PPO/SAC；再横向比较算法；最后把项目按“问题建模、状态动作奖励、算法选择、训练稳定性、指标和失败案例”讲清楚。

核心概念速查

MDP：

$$(S, A, P, R, \gamma)$$

Return：

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Value:

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

Q value:

$$Q_{\pi}(s,a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$$

Advantage:

$$A_{\pi}(s,a) = Q_{\pi}(s,a) - V_{\pi}(s)$$

Bellman expectation:

$$V_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma V_{\pi}(S_{t+1}) \mid S_t=s]$$

Bellman optimality:

$$Q^*(s,a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} Q^*(S_{t+1}, a') \mid S_t=s, A_t=a]$$

TD error:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

Policy Gradient:

$$\nabla J(\theta) = \mathbb{E}[\nabla \log \pi_{\theta}(a|s) A(s,a)]$$

PPO ratio:

$$r_t(\theta) = \pi_{\theta}(a_t|s_t) / \pi_{old}(a_t|s_t)$$

PPO clipped objective:

$$\mathbb{E}[\min(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t)]$$

SAC soft target:

$$y = r + \gamma [\min_i Q_i^-(s', a') - \alpha \log \pi(a'|s')]$$

算法横向对比

算法	类型	on/off-policy	关键点	适用
Policy Iteration	DP	模型已知	evaluation + improvement	小规模 MDP

算法	类型	on/off-policy	关键点	适用
Value Iteration	DP	模型已知	Bellman optimality backup	小规模 MDP
Monte Carlo	value estimation	on-policy 常见	完整 return	episodic task
TD(0)	value estimation	on-policy 常见	bootstrap	在线估值
SARSA	value-based	on-policy	target 用实际 a'	表格控制
Q-learning	value-based	off-policy	target 用 max	表格控制
DQN	value-based	off-policy	replay + target network	离散动作
REINFORCE	policy-based	on-policy	Monte Carlo PG	教学基础
A2C/A3C	actor-critic	on-policy	critic 降方差	离散/连续
PPO	actor-critic	on-policy	clipped objective	通用稳定基线
DDPG	actor-critic	off-policy	deterministic actor	连续动作
TD3	actor-critic	off-policy	double Q + delayed update	连续动作
SAC	actor-critic	off-policy	maximum entropy	连续动作

高频概念问答

1. RL 为什么难？

因为 RL 同时面对 delayed reward、探索与利用、非 i.i.d. 数据、动作影响未来数据分布、credit assignment 和训练稳定性问题。

2. Bellman 方程为什么重要？

Bellman 方程描述了价值函数的递归自洽关系：当前价值等于即时 reward 加未来价值。绝大多数 value-based 和 actor-critic 方法都直接或间接基于 Bellman backup。

3. On-policy 和 off-policy 区别？

On-policy 学习当前采样策略的价值或改进当前策略。Off-policy 用 behavior policy 的数据学习另一个 target policy，样本效率更高，但有分布偏移风险。

4. MC 和 TD 区别？

MC 用完整 episode 的真实 return，不 bootstrap，bias 小但 variance 大。TD 用一步 reward 加下一状态价值估计，能在线更新，variance 小但有 bootstrap bias。

5. SARSA 和 Q-learning 区别？

SARSA 是 on-policy，target 使用实际采样的下一个动作。Q-learning 是 off-policy，target 使用下一状态最大 Q 值。

6. DQN 的两个核心稳定技巧？

Experience replay 和 target network。前者打破样本相关性并复用样本，后者稳定 bootstrap target。

7. Double DQN 解决什么？

缓解 DQN 中 $\max$ 操作造成的 Q 值过估计。它用 online network 选动作，用 target network 评估动作。

8. Policy Gradient 怎么理解？

如果某个动作带来正 advantage，就增加该动作在该状态下的概率；如果 advantage 为负，就降低概率。公式是：

$$\nabla J = \mathbb{E}[\nabla \log \pi(a|s) A(s,a)]$$

9. Baseline 为什么不引入 bias？

因为 baseline 不依赖 action，而：

$$\mathbb{E}_{a \sim \pi}[\nabla \log \pi(a|s)] = 0$$

因此减去 baseline 只改变方差，不改变梯度期望。

10. PPO 为什么常用？

PPO 用 clipped objective 限制新旧策略差异，稳定性好、实现简单、适用面广，是 on-policy actor-critic 中非常常用的工程基线。

11. SAC 为什么适合连续控制？

SAC 是 off-policy，样本效率较高；最大熵目标鼓励探索并提升稳定性；双 Q 网络缓解过估计，因此在连续控制任务中表现稳定。

12. Offline RL 为什么难？

因为不能继续探索收集数据。学习策略可能选择数据集中没覆盖的动作，critic 对这些动作估计不可靠，容易出现 extrapolation error。

面试中算法讲解模板

回答某个算法时，按这个顺序讲：

1. 它解决什么问题？
2. 是 value-based、policy-based 还是 actor-critic？
3. 是 on-policy 还是 off-policy？
4. 核心目标函数或更新公式是什么？
5. 关键 trick 是什么？
6. 相比前一个算法改进在哪里？
7. 局限是什么？

例如讲 DQN：

DQN 是把 Q-learning 扩展到高维状态输入的 deep RL 算法。它用神经网络近似 $Q(s,a)$ ，适合离散动作空间。核心 target 是 $r + \gamma \max_{a'} Q_{\text{target}}(s',a')$ 。为了稳定训练，它使用 replay buffer 打破样本相关性、复用历史经验，并使用 target network 稳定 Bellman target。它的局限是难以处理连续动作空间，并且存在 Q 值过估计问题，后续 Double DQN 对此做了改进。

项目准备建议

面试前建议至少准备一个可讲清楚的 RL 小项目。

优先级：

1. CartPole with DQN。
2. LunarLander with PPO。
3. Pendulum or HalfCheetah with SAC。
4. 简化推荐系统 bandit。
5. RLHF toy example，例如偏好数据训练 reward model。

面试例子索引

准备面试时，建议把算法和例子绑定记忆：

算法/概念	推荐例子	为什么适合
MDP / Bellman	GridWorld	状态、动作、转移、奖励最直观。
DP	已知地图的 GridWorld	可以明确说明“已知环境模型”。
MC	Blackjack	episode 短，完整 return 容易得到。
TD	在线推荐或持续控制	不想等 episode 结束，每步都能更新。
SARSA / Q-learning	Cliff Walking	on-policy 与 off-policy 差异明显。
Bandit	广告推荐冷启动	探索和利用最直接。
Function Approximation	Atari 或推荐系统	状态空间无法枚举。
DQN	CartPole / Atari	离散动作 + Q 网络。
Policy Gradient	连续控制机器人	动作连续，难以枚举 max。
Actor-Critic / GAE	游戏或机器人控制	Actor 选动作，Critic 降低方差。
PPO	RLHF 或 LunarLander	策略更新稳定，工程常用。
DDPG / TD3 / SAC	机械臂、MuJoCo	连续动作控制。
Offline RL	历史推荐日志	只能用离线数据，不能随便探索。
Imitation Learning	自动驾驶示范数据	从专家轨迹学习行为。
RLHF	聊天模型对齐	用人类偏好构造 reward。

回答时不要只说“我用 DQN 做 CartPole”。更好的结构是：

CartPole 是离散动作控制任务，状态低维连续，动作是 left/right，reward 是存活步数。DQN 用 Q 网络输出两个动作价值，配合 replay buffer 和 target network 稳定训练。

项目表达模板

项目背景：
任务建模：
状态空间：
动作空间：
奖励设计：
算法选择：
训练流程：
关键工程细节：
实验指标：
遇到的问题：
改进方向：

CartPole DQN 项目讲法

任务建模：

- State：小车位置、速度、杆角度、角速度。
- Action：向左或向右施加力。
- Reward：每保持一步平衡得到 1。
- Done：杆倾斜过大或小车越界。

算法选择：

- 动作空间离散，适合 DQN。
- 用 MLP 近似 Q 函数。
- epsilon-greedy 探索。
- replay buffer 存储 transition。
- target network 稳定 target。

可以强调的问题：

- epsilon decay 如何设置。
- replay buffer 大小。
- target network 更新频率。
- loss 曲线不稳定是否正常。
- reward 是否能达到环境上限。

LunarLander PPO 项目讲法

任务建模：

- State：位置、速度、角度、角速度、腿部接触信息。
- Action：离散喷气控制。
- Reward：接近着陆点、平稳降落、节省燃料等。

算法选择：

- PPO 稳定且适合 policy optimization。
- 使用 GAE 估计 advantage。
- 使用 entropy bonus 保持探索。
- 使用 clipped objective 限制策略更新。

可以强调：

- PPO 是 on-policy，采样效率不如 DQN/SAC。
- 但训练稳定，调参相对友好。
- advantage normalization 对训练很重要。

推荐系统 Bandit 项目讲法

任务建模：

- State/context：用户画像、上下文、历史行为。
- Action：推荐某个 item 或策略。
- Reward：点击、停留、转化、长期留存。

算法选择：

- 如果动作不影响长期状态，可先用 contextual bandit。
- 可比较 epsilon-greedy、UCB、Thompson Sampling。

注意点：

- 线上探索有风险，需要控制流量。
- reward 延迟和偏差要处理。
- 离线评估有 selection bias。

面试前最终检查清单

- 能手写 Bellman expectation 和 optimality equation。
- 能解释 MC、TD、DP 的区别。
- 能写出 SARSA 和 Q-learning 更新公式。
- 能解释 DQN 两个核心 trick。
- 能解释 Double DQN 为什么缓解过估计。
- 能推导 policy gradient 的 log-derivative trick。
- 能证明 baseline 不引入 bias。
- 能解释 Actor-Critic 和 GAE。

- 能手写 PPO clipped objective。
- 能解释 SAC 的最大熵目标。
- 能说明 offline RL 的 extrapolation error。
- 能讲一个完整 RL 项目。

30 分钟口述复习路线

1. 5 分钟：MDP、return、V/Q、Bellman。
2. 5 分钟：DP、MC、TD、SARSA、Q-learning。
3. 5 分钟：DQN、Double DQN、Dueling、Replay。
4. 5 分钟：Policy Gradient、baseline、Actor-Critic、GAE。
5. 5 分钟：PPO、DDPG、TD3、SAC。
6. 5 分钟：offline RL、RLHF、项目总结。

[章节索引](#)

[← 上一章](#) [下一章 →](#)